Case Study 9: Tuple-Based Employee Shift Conflict Detection

Activity sheet case study #48 | Concepts: Tuples, tuple operations, tuple unpacking, strings (slicing), nested loops, lists

1. Problem Statement

Employee shifts are stored as tuples (employee, day, start time, end time) with times written as HH:MM strings. Write a program that validates each shift, compares the time intervals of every pair of shifts, detects overlapping assignments for the same employee on the same day and lists the employees with conflicting schedules.

2. Objectives

- Store records as tuples and read them with tuple unpacking.
- Convert HH:MM strings to minutes using string slicing and int().
- Detect interval overlap using comparison operators and nested loops.
- Report each conflict with its overlap duration and a list of affected employees.

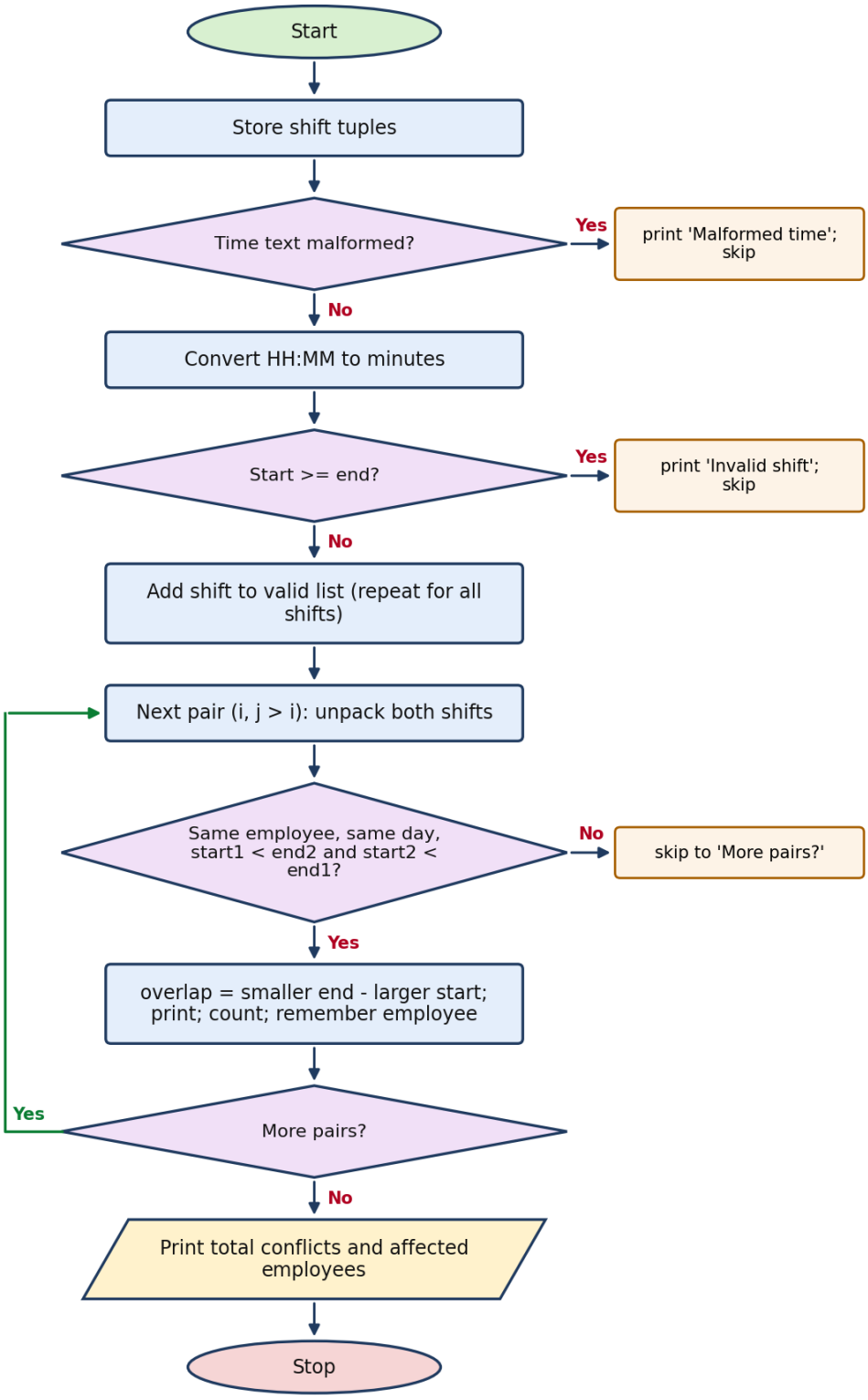
3. Assumptions

- A shift is (employee id, day, 'HH:MM' start, 'HH:MM' end) in 24-hour format; shifts do not cross midnight.
- Two shifts conflict only when they belong to the same employee on the same day and their intervals overlap; shifts that merely touch (one ends when the next starts) do not conflict.
- A shift whose end is not after its start, or whose time text is malformed, is invalid and excluded from conflict checking.

4. Algorithm

1. Store the list of shift tuples.
2. For each shift unpack the fields and check that both times have the form HH:MM with hour <= 23 and minute <= 59; report and skip malformed shifts.
3. Convert the times to minutes after midnight; if start >= end report an invalid shift and skip it.
4. Append the valid shift (employee, day, start, end, texts) to the valid list.
5. For every pair (i, j) with j > i, unpack both shifts.
6. If the employee and day match and start1 < end2 and start2 < end1, compute the overlap = smaller end - larger start, print the conflict, count it and remember the employee.
7. Print the number of conflicts and the employees affected, or a message if there are none.

5. Flowchart



6. Pseudocode

```
START
  valid <- empty list
  FOR each (emp, day, start_text, end_text) in shifts
      IF start_text or end_text is not of the form HH:MM THEN
          PRINT "Malformed time" ; CONTINUE
      start <- HH*60 + MM of start_text ; end <- HH*60 + MM of end_text
      IF start >= end THEN PRINT "Invalid shift" ; CONTINUE
```

```
        APPEND (emp, day, start, end, start_text, end_text) to valid
    END FOR
    conflicts <- 0 ; affected <- empty list
    FOR i = 0 TO LENGTH(valid)-1
        FOR j = i+1 TO LENGTH(valid)-1
            unpack valid[i] and valid[j]
            IF emp1 = emp2 AND day1 = day2 AND start1 < end2 AND start2 < end1 THEN
                overlap <- smaller(end1,end2) - larger(start1,start2)
                PRINT conflict and overlap
                conflicts <- conflicts + 1
                IF emp1 not in affected THEN APPEND emp1
    IF conflicts = 0 THEN PRINT "No scheduling conflicts found"
    ELSE PRINT conflicts and affected
  STOP
```

7. Python Source Code

```
# Case Study 48: Tuple-Based Employee Shift Conflict Detection
# ---- INPUT DATA ----
# (employee, day, start "HH:MM", end "HH:MM")
shifts = [
    ("E101", "Mon", "09:00", "17:00"),
    ("E101", "Mon", "16:00", "20:00"),   # overlaps the first shift
    ("E102", "Mon", "09:00", "13:00"),
    ("E102", "Mon", "13:00", "18:00"),   # touches only: no conflict
    ("E103", "Tue", "08:30", "12:30"),
    ("E103", "Wed", "08:30", "12:30"),   # different day: no conflict
    ("E104", "Tue", "14:00", "10:00"),   # end before start: invalid
    ("E105", "Fri", "06:00", "14:00"),
    ("E105", "Fri", "13:59", "15:00"),   # 1 minute overlap
    ("E105", "Fri", "14:30", "16:00"),   # overlaps the previous shift
    ("E106", "Thu", "9:00", "12:00"),    # malformed time text
]
# ---- END DATA ----

valid_shifts = []
print("SHIFT VALIDATION")
for shift in shifts:
    emp, day, start_text, end_text = shift       # tuple unpacking

    ok = True
    for t in (start_text, end_text):
        if len(t) != 5 or t[2] != ":" or not t[0:2].isdecimal() or not t[3:5].isdecimal():
            ok = False
        elif int(t[0:2]) > 23 or int(t[3:5]) > 59:
            ok = False
    if not ok:
        print("  Malformed time:", emp, day, start_text, end_text)
        continue

    start = int(start_text[0:2]) * 60 + int(start_text[3:5])
    end = int(end_text[0:2]) * 60 + int(end_text[3:5])
    if start >= end:
        print("  Invalid shift (end not after start):", emp, day, start_text, end_text)
        continue
    valid_shifts.append((emp, day, start, end, start_text, end_text))

print("  Valid shifts:", len(valid_shifts), "of", len(shifts))
print()
print("CONFLICT CHECK")
conflicts = 0
affected = []
count = len(valid_shifts)
for i in range(count):
    for j in range(i + 1, count):
        e1, d1, s1, en1, t1a, t1b = valid_shifts[i]
        e2, d2, s2, en2, t2a, t2b = valid_shifts[j]
        if e1 == e2 and d1 == d2 and s1 < en2 and s2 < en1:
            smaller_end = en1
            if en2 < smaller_end:
                smaller_end = en2
            larger_start = s1
            if s2 > larger_start:
                larger_start = s2
            overlap = smaller_end - larger_start
            print("  {} {}: {}-{} overlaps {}-{} by {} min".format(
                  e1, d1, t1a, t1b, t2a, t2b, overlap))
            conflicts = conflicts + 1
            if e1 not in affected:
                affected.append(e1)

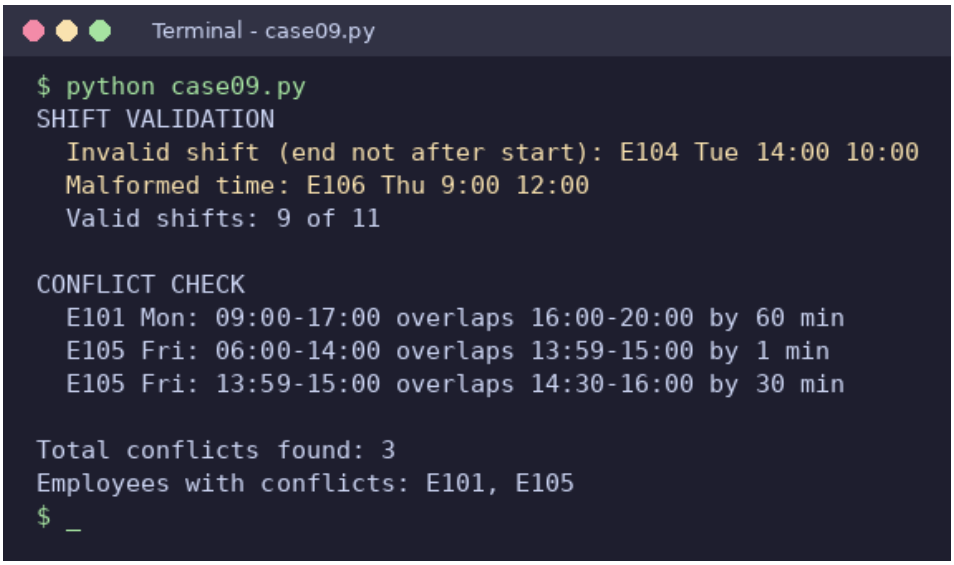
print()
if conflicts == 0:
    print("No scheduling conflicts found.")
else:
    print("Total conflicts found:", conflicts)
    print("Employees with conflicts:", ", ".join(affected))
```

8. Test Cases

#	Test case	Input / data	Expected result	Observed output (actual)	Status
1	Default data	11 shifts, 3 expected conflicts	3 conflicts; E101 and E105	`Total conflicts found: 3 | Employees with conflicts: E101, E105 | E105 Fri: 13:59-15:00 overlaps 14:30-16:00 by 30 min`	**PASS**
2	Touching shifts are not conflicts	09:00-13:00 and 13:00-18:00	Boundary equality allowed	`No scheduling conflicts found.`	**PASS**
3	Same hours on different days	Mon and Tue	Different day	`No scheduling conflicts found.`	**PASS**
4	Different employees, same time	A and B at same time	Different employee	`No scheduling conflicts found.`	**PASS**
5	One shift inside another	09-17 contains 10-11	Full containment is a conflict	`A Mon: 09:00-17:00 overlaps 10:00-11:00 by 60 min`	**PASS**
6	End before start	14:00 to 10:00	Rejected	`Invalid shift (end not after start): A Mon 14:00 10:00 | Valid shifts: 0 of 1`	**PASS**
7	Malformed and out-of-range times	9:00, 25:00, 10:75	Rejected	`Malformed time: A Mon 9:00 10:00 | Malformed time: B Mon 25:00 26:00 | Malformed time: C Mon 10:75 11:00`	**PASS**
8	No shifts	[]	No crash	`No scheduling conflicts found. | Valid shifts: 0 of 0`	**PASS**

9. Output Screenshot

Terminal output of `python case09.py` using the default data:



10. Conclusion

Representing shifts as tuples keeps each record compact and immutable, and unpacking makes the comparison code readable. Converting times to minutes turns interval overlap into two simple comparisons, and the tests confirm that touching shifts, different days and different employees are correctly ignored while true overlaps are reported with their duration.